

Dynamics Processing

Outline

- History
- Digital Compressor Design
 - Level Detector
 - Gain Computer
 - Compressor Architectures
- Design challenges

History

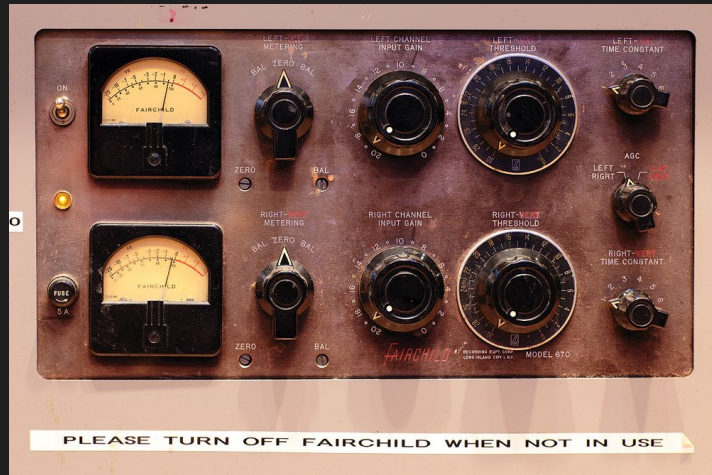
- Problem:
 - Human voices talking into microphones
 - Wide dynamic range!
 - Whisper vs. shout
 - Close to mic vs. far
 - How to amplify evenly
 - PA announcement
 - Telephone signals
- Early examples
 - Telefunken U3 (1936)
 - Western Electric 110A (1937)



History

- How it works
- Reduce dynamic range
 - Amplify “quiet” parts of the sound
 - Attenuate “loud” parts of the sound
- Mechanisms
 - Vacuum Tubes
 - Vactrol (light-dependent resistor)
 - Field-Effect Transistors
 - Voltage-Controlled Amplifier (VCA)

Tube Compressors (Vari-Mu)



Opto Compressors



FET Compressors



VCA Compressors



Other Effects

- Compression is just one type of dynamics processing
- Expander
 - “Expand” the dynamic range instead of compressing
 - Useful for accentuating transients or quieting background noise
- Gate
 - “Silence” the quiet parts of the sound
- Limiter
 - Similar to compression, compresses dynamic range into a level “ceiling”

Digital Compressor Design

Reference:

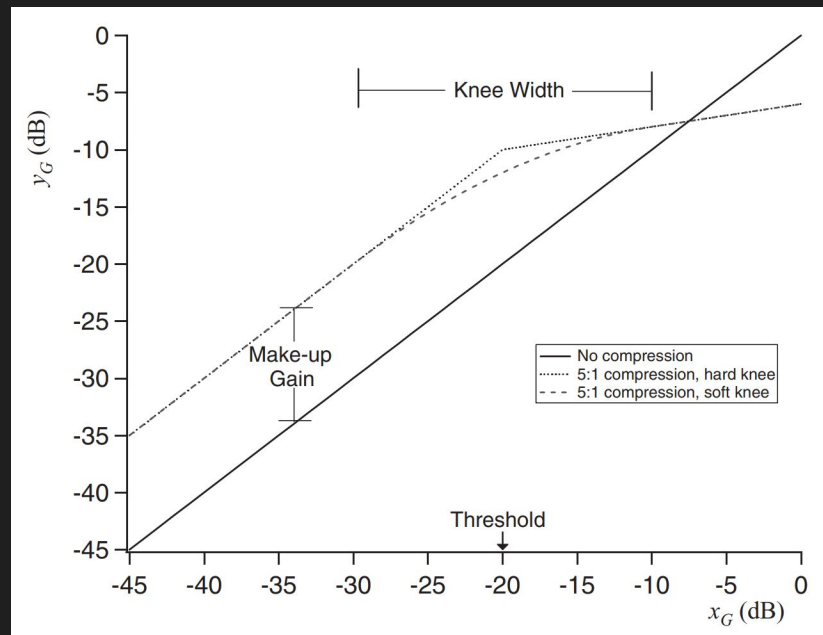
“Digital Dynamic Range Compressor Design - A Tutorial and Analysis”

Giannoulis, Massberg, and Reiss

Journal of the AES, 2012

Compressor Basics

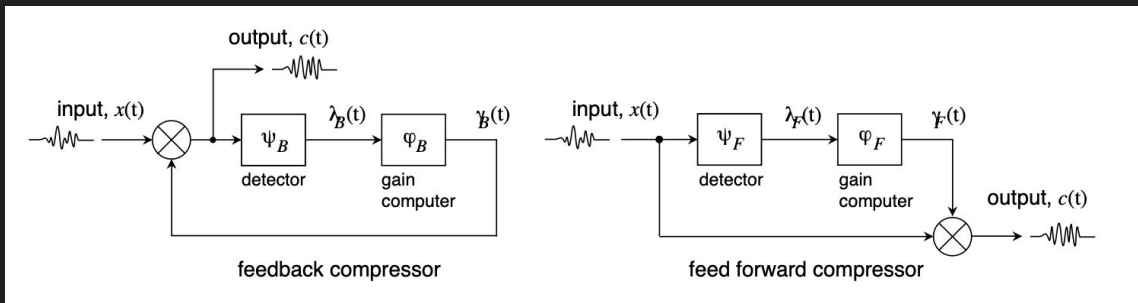
- Threshold
 - At what loudness does compression start?
- Ratio
 - How many dB do we attenuate by?
 - As a fraction of the dB over the threshold
- Knee
 - dB range (around the threshold)
 - Rounds the “corner” of the threshold
- Makeup
 - Extra gain to offset the attenuation



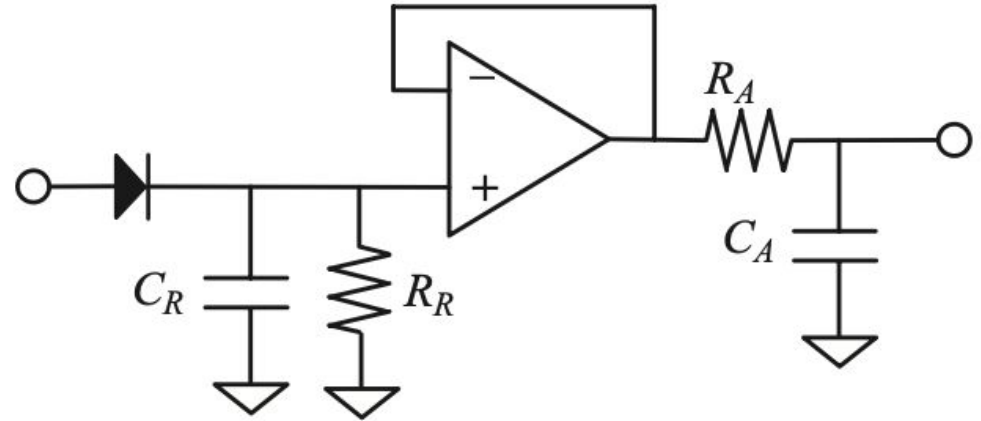
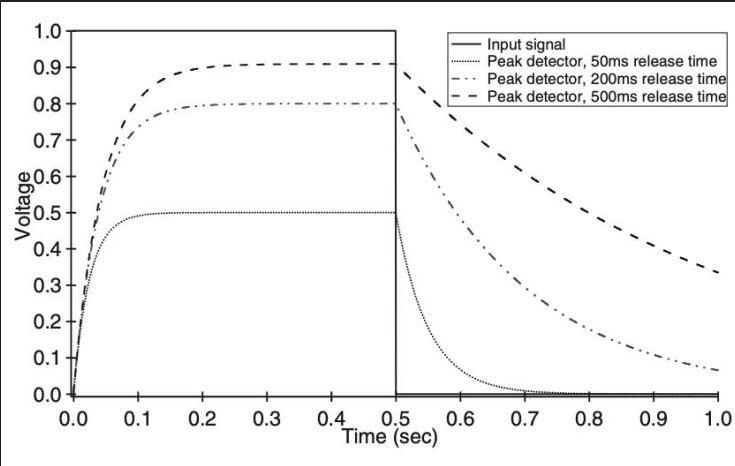
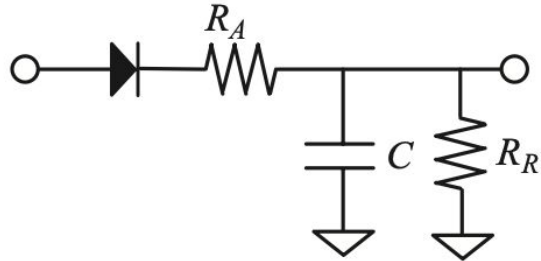
Compressor Basics

In order to design a compressor, we need two things:

- A “level detector”
 - Estimate the current level of the signal at any point in time
- A “gain computer”
 - Compute the required amount of attenuation (or amplification) based on current level estimate
- Architectures:
 - Feed-forward vs. Feed-back?
 - Detector before/after gain computer?



Level Detector



Level Detector

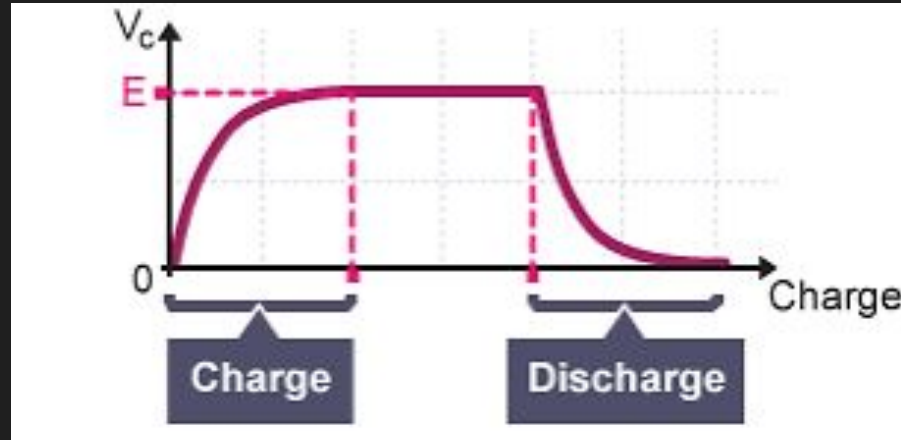
"Rectified" signal:

- Absolute: $x_{rect} = |x|$
- Half-wave rectified: $x_{rect} = \max(x, 0)$
- Square: $x_{rect} = x^2$
- Diode model?

Level Detector

Capacitor charging/discharging → Exponential step response:

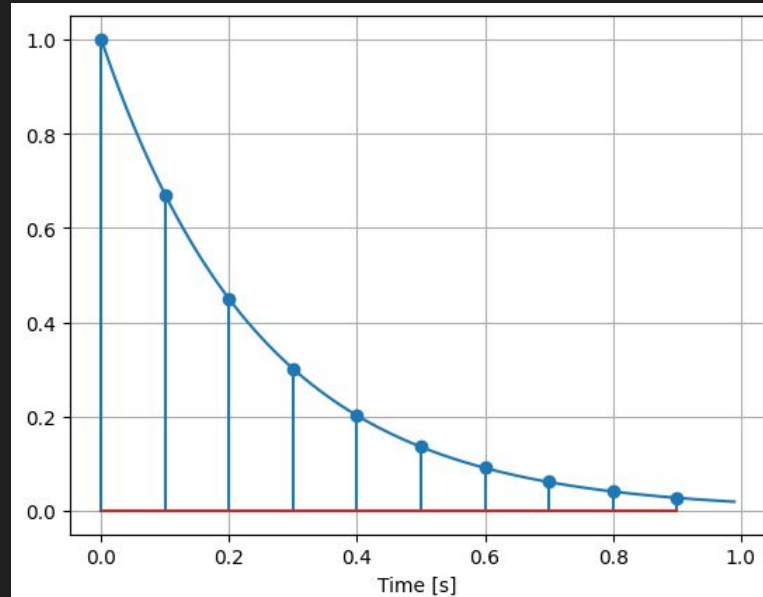
$$y(t) = e^{(-t/\tau)}$$



Level Detector

Impulse invariance: $t = n/T = n * f_s$

$$y[n] = e^{-\frac{nf_s}{\tau}}$$



Level Detector

Recursive definition:

$$y[n] = e^{-\frac{nf_s}{\tau}}$$

$$y[n-1] = e^{-\frac{(n-1)f_s}{\tau}}$$

$$y[n-1] = e^{-\frac{nf_s}{\tau}} e^{\frac{f_s}{\tau}}$$

$$y[n-1] = y[n] * e^{\frac{f_s}{\tau}}$$

$$y[n] = y[n-1] * e^{-\frac{f_s}{\tau}}$$

Level Detector

Peak detector:

$$a := e^{-\frac{f_s}{\tau}}$$
$$y[n] = b * x_{rect}[n] + a * y[n - 1]$$

For "unity" gain, set $b = 1 - a$

Switched attack/release:

$$y[n] = \begin{cases} b_A * x_{rect}[n] + a_A * y[n - 1] & \text{if } x_{rect}[n] > y[n - 1] \\ b_R * x_{rect}[n] + a_R * y[n - 1] & \text{if } x_{rect}[n] \leq y[n - 1] \end{cases}$$

Level Detector

```
struct Level_Detector
{
    float y_n1 {};
    float fs { 48000.0f };

    void process (float* data, int N, float attack_s, float release_s)
    {
        const auto a_att = std::exp(-fs / attack_s);
        const auto b_att = 1.0f - a_att;
        const auto a_rel = std::exp(-fs / release_s);
        const auto b_rel = 1.0f - a_rel;

        for (int n = 0; n < N; ++n)
        {
            auto x_rect = std::abs (data[n]);

            auto is_attack = x_rect > y_n1;
            auto b_coeff = is_attack ? b_att : b_rel;

            // auto y_n = b_coeff * x_rect + a_coeff * y_n1;

            // re-written with a single coefficient
            auto y_n = b_coeff * (x_rect - y_n1) + y_n1;

            data[n] = y_n;
        }
    }
};
```

Gain Computer

$$y_G = \begin{cases} x_G & x_G \leq T \\ T + (x_G - T)/R & x_G > T \end{cases}$$

- Case 1: Linear
 - Level is below threshold
 - Output level is the same as input
- Case 2: Compression
 - Level is above threshold
 - Output is attenuated according to ratio

Gain Computer

$$y_G = \begin{cases} x_G & 2(x_G - T) < -W \\ x_G + (1/R - 1)(x_G - T + W/2)^2/(2W) & 2|(x_G - T)| \leq W \\ T + (x_G - T)/R & 2(x_G - T) > W \end{cases}$$

- If level is in the “knee” range:
 - Quadratic fit between top of linear range and bottom of compression range
 - Other fits are also acceptable
 - Exponential
 - Higher-order polynomial

Gain Computer

Computation in Decibels vs. linear gain

$$\ell_O = \ell_T + (\ell_I - \ell_T)/\rho$$

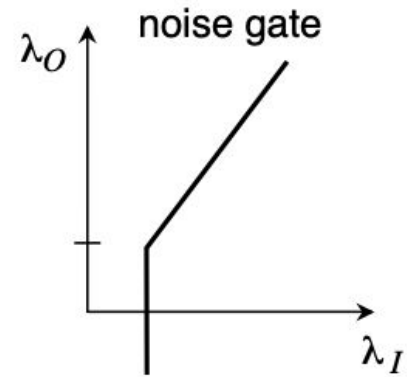
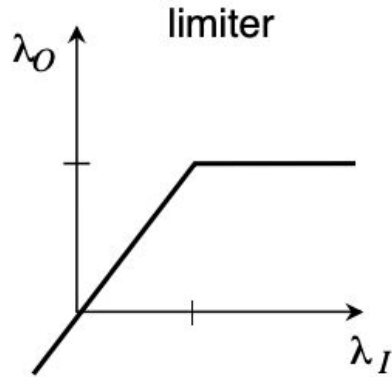
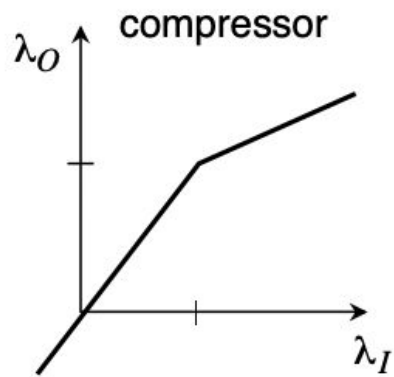
$$g(\ell_I) = \ell_O - \ell_I = (\ell_I - \ell_T)/\rho + \ell_T - \ell_I$$

$$g(\ell_I) = (\ell_I - \ell_T)(1/\rho - 1)$$

$$\Phi_F = \left(\frac{\lambda_I}{\lambda_T} \right)^{(1/\rho - 1)}$$

Gain Computer

Gain computer shapes for other dynamic effects:



Gain Computer

```
void (const float* level, float* gain, int N)
{
    for (int n = 0; n < N; ++n)
    {
        const auto level_db = gain2db(level);

        float gain_db;

        if (level_db <= threshold_db) // linear range
            gain_db = 0.0f; // no gain change

        else // compression range
            gain_db = threshold_db + (level_db - threshold_db) / ratio;

        gain[n] = db2gain(gain_db);
    }
}
```


Gain Computer

```
void (const float* level, float* gain, int N)
{
    for (int n = 0; n < N; ++n)
    {
        if (level[n] <= threshold) // linear range
            gain[n] = 1.0f;

        else // compression range
            gain[n] = std::pow (level[n] / threshold, (1.0f / ratio) - 1.0f);
    }
}
```

Gain Computer

```
void (const float* level, float* gain, int N)
{
    const auto half_knee = knee_db * 0.5f;
    const auto alpha = (1.0f - (1.0f / ratio)) / (2.0f * knee_db);

    for (int n = 0; n < N; ++n)
    {
        const auto level_db = gain2db(level);

        float gain_db;

        if (level_db <= threshold_db - half_knee) // linear range
            gain_db = 0.0f; // no gain change

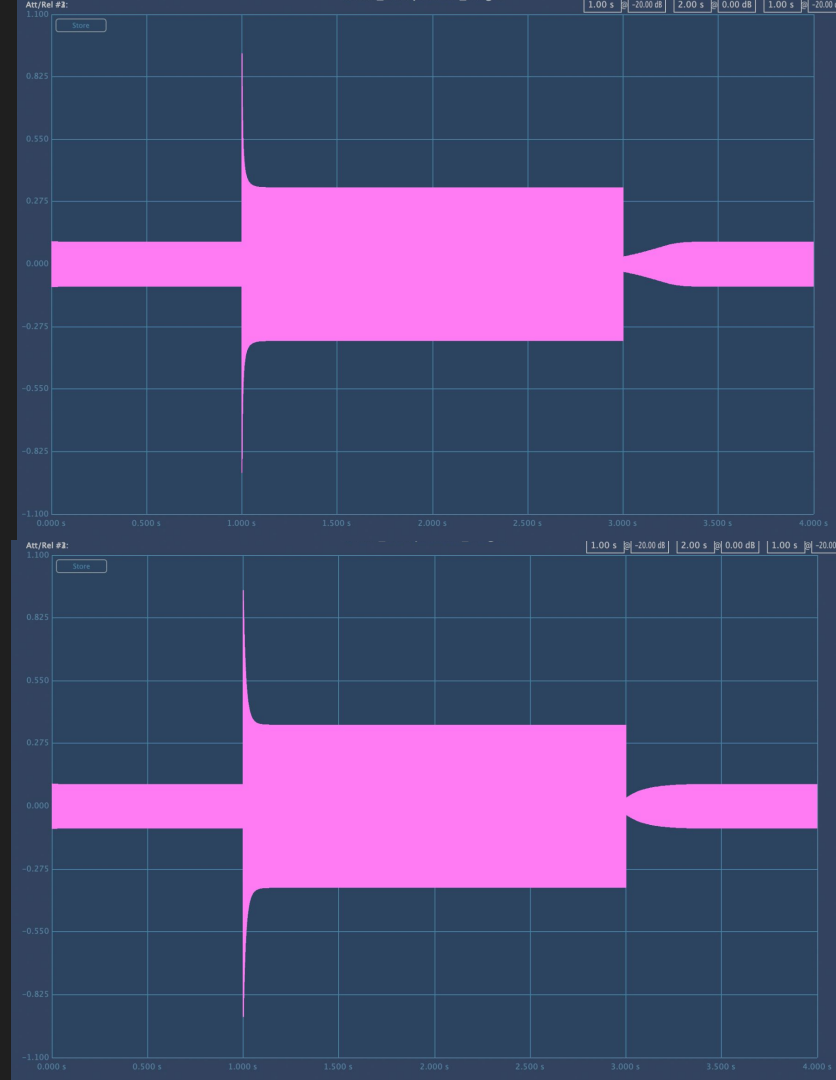
        else if (level_db > threshold_db + half_knee) // compression range
            gain_db = threshold_db + (level_db - threshold_db) / ratio;

        else // knee range
        {
            auto gc = level_db - threshold_db + half_knee;
            gain_db = -1.0f * alpha * gc * gc;
        }

        gain[n] = db2gain(gain_db);
    }
}
```

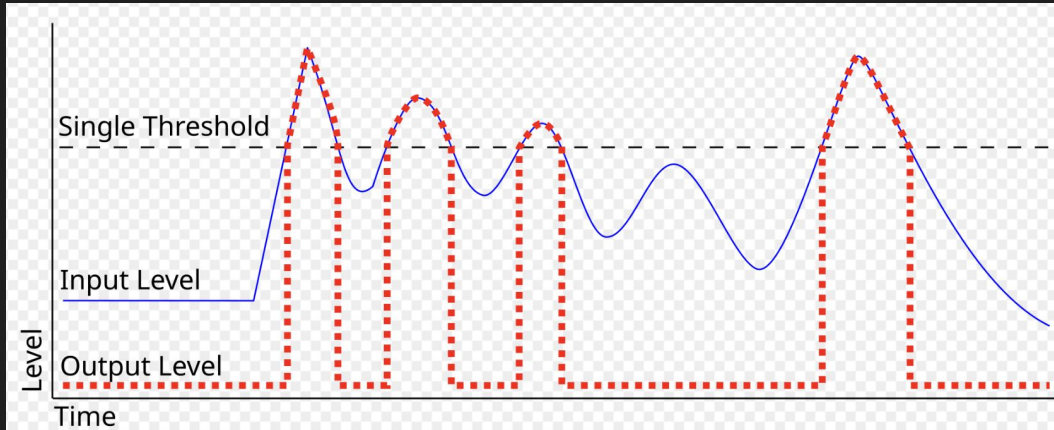
Variations

- Level Detector
 - Auto-Release
 - Program-dependent release
 - Multi-stage release
 - “Release to zero” vs. “Release to threshold”
 - “Hold” stage
 - Very useful for Gate effects
- Gain Computer
 - Alternate “knee” shapes
 - “Range”: limit the max amount of attenuation
- Architecture
 - Level Detector → Gain Computer
 - Gain Computer → Level Detector



Design Challenges

- Harmonic Distortion
 - Aliasing
- “Chatter”
- “Pumping” (can be good or bad)



Exercise: Multi-Band Compressor